Studio Nx

P1-08 회원 정보 API — 함수 설명 합본 (35~43)

2026-10-12

목차

1. ai-notes/backend(fastAPI_Python)/P1-08 - 회원 정보 API/35_user_service_profile_image_url.md
2. ai-notes/backend(fastAPI_Python)/P1-08 - 회원 정보 API/36_user_service_to_me_response.md
3. ai-notes/backend(fastAPI_Python)/P1-08 - 회원 정보 API/37_user_service_update_me.md
4. ai-notes/backend(fastAPI_Python)/P1-08 - 회원 정보 API/38_user_service_save_profile_image.md
5. ai-notes/backend(fastAPI_Python)/P1-08 - 회원 정보 API/39_user_service_delete_profile_image.md
6. ai-notes/backend(fastAPI_Python)/P1-08 - 회원 정보 API/40_user_service_profile_image_file.md
7. ai-notes/backend(fastAPI_Python)/P1-08 - 회원 정보 API/41_users_route_me.md
8. ai-notes/backend(fastAPI_Python)/P1-08 - 회원 정보 API/42_users_route_profile_image.md
9. ai-notes/backend(fastAPI_Python)/P1-08 - 회원 정보 API/43_users_route_profile_image_file.md

Backend(FastAPI/Python) 35 — `profile_image_url()`

항목	내용
파일	`studionx/backend/app/services/user_service.py`
작업	P1-08 회원 정보: 프로필 사진 주소 (PRD 01 FR-106)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	34 `POST /auth/logout` · 36 `to_me_response()`

1. 하는 일

회원 프로필 사진의 **저장 경로**를, 화면이 `<img src>`에 넣을 **주소**로 바꿉니다. 사진이 없으면 `None`을 돌려줍니다.

`user.profile_image_path` (DB)	돌려주는 값
`None`, `""`	`None`입니다. 화면은 대신 닉네임의 첫 글자를 보여 줍니다
`"profiles/3f9c2b7e0d1a4c5b8e6f7a9b0c1d2e3f.jpg"`	`"/api/v1/profile-images/3f9c2b7e0d1a4c5b8e6f7a9b0c1d2e3f.jpg"`

2. 언제 쓰나

쓰는 곳	하는 일
36번 `to_me_response()`	`GET /users/me` 등의 응답에 들어가는 `profile_image_url` 칸
(P1-16) 대시보드 응답	프로필 카드의 사진

사용자 결정 (2026-10-07): 주소에 회원 번호를 넣지 않음

템플릿의 주소는 `/api/v1/users/{회원 번호}/profile-image?v=…`였습니다. 이 주소는 로그인 없이도 열립니다(`<img>`에는 토큰을 붙일 수 없기 때문입니다). 그러면 누구나 번호를 1, 2, 3… 바꿔 가며 **그 번호의 회원이 있는지, 사진이 무엇인지** 볼 수 있습니다. 이것은 P1-04 때 정한 **탈퇴 추적 방지 ③ "회원 번호를 받는 공개 API는 만들지 않는다"**와 어긋납니다. 그래서 사용자가 **무작위 파일 이름** 방식을 골랐습니다.

	템플릿	결정
저장 경로	`profiles/{회원 번호}/{무작위}.jpg`	`profiles/{무작위 32자}.jpg`
주소	`/api/v1/users/85/profile-image?v=1696651200`	`/api/v1/profile-images/{무작위 32자}.jpg`
남의 사진을 찾아보려면	번호만 바꾸면 됩니다	32자(16진수, 2^{128}가지)를 맞혀야 합니다
사진을 바꿨을 때 캐시 문제	`?v=`로 해결합니다	파일 이름 자체가 바뀌어서 필요 없습니다

함께 고친 문서와 주석은 PRD 01 §7의 API 표, 이 파일의 사진 저장(38)과 사진 파일(40) 템플릿 주석, `users.py`의 머리말입니다. 라우터 주소 자체는 43번을 구현할 때 바꿉니다.

3. 코드

import (파일 맨 위)

```
from pathlib import Path, PurePosixPath

from fastapi import UploadFile
from sqlalchemy.orm import Session

from app.models import User
from app.schemas.user import MeResponse, MeUpdateRequest
```

`PurePosixPath`를 이번에 추가했습니다.

상수 (이번에 추가)

```
# 사진을 두는 폴더 (settings.storage_dir 아래) · 화면이 사진을 받는 주소 앞부분
PROFILE_DIR = "profiles"
PROFILE_IMAGE_URL = "/api/v1/profile-images/"
```

함수

```
def profile_image_url(user: User) -> str | None:
    if not user.profile_image_path:
        return None
    return PROFILE_IMAGE_URL + PurePosixPath(user.profile_image_path).name
```

문법 풀이

import 문법	뜻
`from pathlib import Path, PurePosixPath`	`Path`는 실제 파일(디스크)을 다룰 때 쓰고, `PurePosixPath`는 경로를 **글자로만** 다룹니다. 디스크를 보지 않으며, 구분자는 항상 `/`입니다

식	뜻
`-> str | None`	문자열 또는 `None`을 돌려준다는 표시입니다 (Python 3.10 문법)
`if not user.profile_image_path:`	`None`과 빈 문자열 `""`를 함께 걸러 냅니다
`PurePosixPath("profiles/3f9c….jpg").name`	경로의 마지막 부분, 즉 파일 이름 `"3f9c….jpg"`입니다
`PROFILE_IMAGE_URL + …`	문자열을 이어 붙입니다

4. 흐름

1. 저장 경로가 없으면 `None`을 돌려줍니다.
2. 있으면 파일 이름만 꺼내서 주소의 앞부분에 붙여 돌려줍니다.

5. 왜 이렇게

선택	이유
주소에는 파일 이름만 넣음 (폴더는 뺌)	폴더 이름(`profiles`)은 서버 내부 사정입니다. 주소는 `/profile-images/`만으로도 사진이라는 것을 나타냅니다
`Path`가 아니라 `PurePosixPath` 사용	DB의 경로는 항상 `/`로 저장합니다 (DB.md §1.5 상대 경로). Windows에서 `Path`를 쓰면 구분자가 `\`로 바뀔 수 있습니다. 게다가 디스크를 볼 필요도 없습니다
문자열 자르기(`split('/')[-1]`) 대신 `PurePosixPath`	무엇을 하는지가 이름에 드러납니다
주소 앞부분을 상수로 둠	40번과 43번(사진 파일 라우터)이 같은 주소를 씁니다. 한 곳만 고치면 됩니다
DB를 보지 않음	이미 가진 `User`만으로 계산합니다. 응답을 만들 때마다 부르는 함수라서 가볍게 만들었습니다

6. 시험 (`tests/test_user_service.py` 새 파일, 5개)

시험	확인한 것
사진 없음(`None`), 빈 문자열	`None`을 돌려줍니다
`profiles/{32자}.jpg`, `.webp`	`/api/v1/profile-images/{32자}.jpg`, `.webp`가 됩니다
회원 번호 85	주소에 `85`와 `/users/`가 들어 있지 않습니다

전체 시험 239개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 36 — `to_me_response()`

항목	내용
파일	`studionx/backend/app/services/user_service.py`
작업	P1-08 회원 정보: 내 정보 응답 만들기 (PRD 01 FR-106)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	35 `profile_image_url()` · 37 `update_me()`

1. 하는 일

DB의 회원 한 행(`User`)을 화면에 보낼 **내 정보 응답(`MeResponse`)**으로 바꿉니다. 미리 정해 둔 7칸만 담습니다.

```
{
  "user_id": 85,
  "email": "demo@example.com",
  "nickname": "demo_user",
  "role": "USER",
  "profile_image_url": "/api/v1/profile-images/3f9c….jpg",
  "created_at": "2026-10-07T01:12:30",
  "last_login_at": "2026-10-07T09:30:00"
}
```

담지 않는 칸 (User에는 있음)	이유
`password_hash`	해시라도 밖으로 나가면 안 됩니다. 무작위 대입으로 원래 비밀번호를 찾아내는 데 쓰일 수 있습니다
`status`, `deleted_at`	이 응답은 로그인한 **활동 회원**만 받습니다 (06번이 이미 막습니다)
`privacy_agreed_at`, `privacy_version`, `storage_*` 등	회원 정보 화면이 쓰지 않는 칸입니다. 용량은 P1-16 대시보드 응답이 따로 보내 줍니다

2. 언제 쓰나

쓰는 곳 (P1-08 라우터)	
`GET /users/me` (41)	내 정보를 보낼 때
`PATCH /users/me` (41)	닉네임을 바꾼 뒤 바뀐 정보를 보낼 때
`POST`, `DELETE /users/me/profile-image` (42)	사진을 바꾸거나 지운 뒤 바뀐 정보(새 `profile_image_url`)를 보낼 때

화면에서는 `userinfo.js`의 `loadMe`(P1-10)와 `mypage.js`가 씁니다.

응답에 `user_id`를 넣어도 괜찮습니다. 이 응답은 **토큰을 가진 본인**에게만 가기 때문입니다. 탈퇴 추적 방지 ③은 "남의 번호로 조회하는 공개 API"를 막는 규칙입니다.

3. 코드

import (파일 맨 위: 35번과 같고, 새로 추가한 것은 없습니다)

```
from pathlib import Path, PurePosixPath

from fastapi import UploadFile
from sqlalchemy.orm import Session

from app.models import User
from app.schemas.user import MeResponse, MeUpdateRequest
```

이 함수는 **`User`, `MeResponse`**와 35번 `profile_image_url`을 사용합니다.

함수

```
def to_me_response(user: User) -> MeResponse:
    return MeResponse(
        user_id=user.user_id,
        email=user.email,
        nickname=user.nickname,
        role=user.role,
        profile_image_url=profile_image_url(user),
        created_at=user.created_at,
        last_login_at=user.last_login_at,
    )
```

문법 풀이

import 문법	뜻
`from app.schemas.user import MeResponse`	템플릿에 있던 응답 모양입니다 (pydantic `BaseModel`)

식	뜻
`MeResponse(user_id=…, …)`	이름을 붙인 인자(키워드 인자)로 칸마다 값을 넣어 만듭니다. 칸이 빠졌거나 형식이 틀리면 pydantic이 바로 오류를 냅니다
`role=user.role`	DB 값은 `UserRole.USER`(StrEnum)입니다. `StrEnum`은 문자열이기도 해서 응답 칸 `str`에 그대로 들어가고, JSON에는 `"USER"`로 나갑니다
`created_at` (`datetime`)	FastAPI가 JSON으로 바꿀 때 `"2026-10-07T01:12:30"` 모양의 문자열로 만듭니다

4. 흐름

1. 회원의 칸 6개를 그대로 옮깁니다.
2. 사진은 35번으로 주소를 만듭니다.
3. 응답 객체를 돌려줍니다.

5. 왜 이렇게

선택	이유
칸을 **하나씩** 적음	`MeResponse.model_validate(user)`(ORM 객체에서 자동으로 꺼내기)를 쓸 수도 있습니다. 하지만 그러려면 스키마에 `from_attributes` 설정이 필요하고, `profile_image_url`은 DB 칸이 아니라 따로 계산해야 합니다. 하나씩 적어 두면 무엇이 나가는지 한눈에 보입니다

선택	이유
응답 칸을 스키마로 고정	나중에 `User`에 칸이 늘어나도 응답에 저절로 섞이지 않습니다. 시험이 "칸은 7개뿐"인지 확인합니다
DB를 다시 읽지 않음	부르는 쪽이 이미 최신 `User`를 가지고 있습니다 (수정한 뒤에는 37번과 38번이 `refresh`합니다)

6. 시험 (`tests/test_user_service.py`에 3개를 추가해서 모두 8개, 실제 MySQL로 실행)

시험	확인한 것
새 회원	7칸의 값이 DB와 같고, `role`은 `"USER"`, 사진과 마지막 로그인은 `None`이며, `created_at`은 DB가 채운 값입니다
응답 칸	정확히 7개입니다. JSON 안에 비밀번호 해시 문자열과 `password`가 없습니다
관리자, 사진, 로그인 기록	`role`은 `"ADMIN"`, `last_login_at`은 그대로, 사진 주소는 `/api/v1/profile-images/{32자}.png`입니다

전체 시험 242개가 통과했고, `ruff` 검사도 통과했습니다. 시험에서 만든 회원은 시험이 끝나면 되돌려집니다 (conftest의 savepoint).

Backend(FastAPI/Python) 37 — `update_me()` (+ `MeUpdateRequest`)

항목	내용
파일	`studionx/backend/app/services/user_service.py`, `app/schemas/user.py`
작업	P1-08 회원 정보: 닉네임 변경 (PRD 01 FR-106)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	36 `to_me_response()` · 38 `save_profile_image()`

1. 하는 일

내 닉네임을 바꿉니다. 가입할 때와 같은 규칙(2~20자, 다른 사람과 겹치지 않음, 탈퇴 3일 규칙)을 지킵니다. 이메일은 바꿀 수 없습니다 (FR-106).

지금 닉네임 → 보낸 값	결과
`my_name` → (칸 없음), `null`, `"my_name"`	아무것도 하지 않습니다
`old_name` → `" New_Name "`	앞뒤 공백을 지운 `New_Name`으로 저장합니다
`Demo_Case` → `"demo_case"` (대소문자만 다름)	저장합니다. 내 닉네임이기 때문입니다
`my_name` → `"taken_name"` (다른 사람이 `Taken_Name`을 쓰는 중)	409 `NICKNAME_TAKEN` "This nickname is already in use."
`my_name` → `"gone_nick"` (어제 탈퇴한 회원의 닉네임)	409 `REJOIN_TOO_SOON` "This nickname can be used again after Oct 9, 2026 (UTC)."
`my_name` → `"free_again"` (4일 전에 탈퇴한 회원의 닉네임)	저장합니다. 탈퇴 회원 쪽 닉네임은 `deleted_user_0000000085`로 비켜 둡니다 (P1-04 `release_withdrawn`)
`" a "`, 21자	422입니다. 스키마에서 막힙니다 (아래 설명)

2. 언제 쓰나

쓰는 곳	
`PATCH /users/me` (41)	본문 `{"nickname": "…"}`를 받아 이 함수를 부르고, 36번으로 응답을 만듭니다
화면	userInfo.html의 닉네임 칸, Check 버튼, Save changes (P1-10)

템플릿에서 바꾼 것 2가지

	템플릿	바꾼 것	이유
스키마 `MeUpdateRequest.nickname`	`str`과 길이 2~20	가입과 같은 **`Nickname`** 타입	템플릿대로라면 `" a "`가 5자로 통과하고, 저장할 때 공백을 지우면 1자짜리 닉네임이 생깁니다. `Nickname`은 공백을 먼저 지운 뒤 길이를 잽니다
"지금 닉네임과 다르면 확인"	글자가 다르면 확인	**DB가 같다고 판단하면** 확인을 건너뜀	DB의 비교 규칙(utf8mb4_0900_ai_ci)은 대소문자를 구분하지 않습니다. 그래서 `Demo_Case` → `demo_case`를 확인하면 **내 닉네임**이 "사용 중"으로 나와 바꿀 수 없게 됩니다

3. 코드

import (파일 맨 위)

```
from pathlib import Path, PurePosixPath

from fastapi import UploadFile
from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import User
from app.schemas.user import MeResponse, MeUpdateRequest
from app.services.auth_service import check_nickname
```

`select`, `IntegrityError`, `AppError`, `ErrorCode`, `check_nickname`을 이번에 추가했습니다.

스키마 (`schemas/user.py`)

```
from app.schemas.auth import Nickname

class MeUpdateRequest(BaseModel):
    nickname: Nickname | None = None
```

함수

```
def update_me(db: Session, user: User, data: MeUpdateRequest) -> User:
    nickname = data.model_dump(exclude_unset=True).get("nickname")
    if nickname is None or nickname == user.nickname:
        return user

    mine = db.scalar(
        select(User.user_id).where(User.user_id == user.user_id, User.nickname == nickname)
    )
    if mine is None:
        result = check_nickname(db, nickname, release=True)
        if result.reason == "REJOIN_TOO_SOON":
            when = result.available_at
            raise AppError(
                ErrorCode.REJOIN_TOO_SOON,
                409,
                f"This nickname can be used again after {when:%b} {when.day}, {when.year} (UTC).",
                details={"nickname": "REJOIN_TOO_SOON", "available_at": when.isoformat()},
            )
        if not result.available:
            raise AppError(ErrorCode.NICKNAME_TAKEN, 409, "This nickname is already in use.")

    user.nickname = nickname
    try:
        db.commit()
    except IntegrityError:
        # 확인과 저장 사이에 다른 사람이 같은 닉네임으로 먼저 바꾼 경우
        db.rollback()
        raise AppError(ErrorCode.NICKNAME_TAKEN, 409, "This nickname is already in use.") from None
```

```
    db.refresh(user)
    return user
```

문법 풀이

import 문법	뜻
`from app.services.auth_service import check_nickname`	P1-04(19)의 닉네임 확인 함수를 그대로 씁니다
`from sqlalchemy.exc import IntegrityError`	UNIQUE 같은 DB 규칙을 어겼을 때 나는 오류입니다

식	뜻
`Nickname | None = None`	닉네임 타입이거나 `None`이고, 보내지 않으면 `None`입니다
`data.model_dump(exclude_unset=True)`	**보낸 칸만** 사전으로 꺼냅니다. 보내지 않은 칸은 빠지므로, 나중에 칸이 늘어나도 보내지 않은 칸은 바뀌지 않습니다
`.get("nickname")`	칸이 없으면 `None`입니다
`select(User.user_id).where(A, B)`	`WHERE A AND B`입니다. 그 회원이면서, DB 규칙상 닉네임이 같은 행을 찾습니다
`db.scalar(…)`	첫 행의 첫 값을 가져오고, 없으면 `None`입니다
`f"{when:%b}"`	날짜를 `Oct`처럼 달 이름으로 씁니다
`raise … from None`	원래의 DB 오류(IntegrityError)를 오류 기록에 붙이지 않습니다. 예상한 상황이기 때문입니다

4. 흐름

1. 보낸 닉네임을 꺼냅니다. 없거나 지금 닉네임과 똑같으면 그대로 돌려줍니다.
2. DB가 "내 닉네임과 같다"고 판단하는지 확인합니다 (대소문자나 악센트만 다른 경우).
3. 같지 않다면 P1-04의 닉네임 확인을 거칩니다. 탈퇴한 지 3일이 안 된 닉네임이면 409 `REJOIN_TOO_SOON`, 사용 중이면 409 `NICKNAME_TAKEN`으로 응답합니다. (3일이 지난 탈퇴 닉네임은 이 안에서 비켜 둡니다.)
4. 저장합니다. 그사이 다른 사람이 먼저 가져갔다면(UNIQUE 위반) 되돌리고 409로 응답합니다.
5. DB에서 다시 읽어서 돌려줍니다.

5. 왜 이렇게

선택	이유
똑같으면 DB를 건드리지 않음	화면에서 닉네임을 바꾸지 않고 Save를 눌러도 불필요한 쓰기와 확인이 일어나지 않습니다
`null`도 "바꾸지 않음"으로 처리	닉네임은 비울 수 없습니다 (DB `NOT NULL`). 오류를 내는 것보다 무시하는 편이 안전합니다
탈퇴 3일 문구를 따로 만듦	P1-04의 `unavailable_error`는 "You can **sign up** with this nickname again after…"처럼 가입용 문구입니다. 이미 검토가 끝난 함수라 고치지 않고, 여기서 수정 화면에 맞는 문구를 만들었습니다
`release=True`	3일이 지난 탈퇴 닉네임은 가입할 때처럼 바로 쓸 수 있어야 합니다 (FR-108). 비켜 두는 변경은 같은 `commit`으로 함께 저장되고, 409로 끝나면 저장하지 않으므로 함께 취소됩니다
`IntegrityError`를 409로 바꿈	P1-04 가입과 같은 이유입니다. 확인과 저장 사이의 짧은 틈을 막습니다
마지막에 `refresh`	DB가 바꾼 값(`updated_at`)까지 받아서 36번 응답에 씁니다

6. 시험 (`tests/test_user_service.py`에 14개를 추가해서 모두 22개, 실제 MySQL로 실행)

시험	확인한 것
스키마	`" ab "`는 `"ab"`가 되고 20자는 통과합니다. `" a "`, 21자, 빈 문자열은 오류입니다
바꿀 것이 없음 (칸 없음, `null`, 같은 값)	같은 객체를 돌려주고, DB는 그대로입니다
새 닉네임 `" New_Name "`	DB에 `New_Name`이 저장됩니다
대소문자만 다름 (`Demo_Case` → `demo_case`)	저장됩니다
다른 사람의 닉네임 (대소문자만 다름)	409 `NICKNAME_TAKEN`이고, 내 닉네임은 그대로입니다
어제 탈퇴한 회원의 닉네임	409 `REJOIN_TOO_SOON`과 문구, `details`가 나오고, 내 닉네임은 그대로입니다
4일 전에 탈퇴한 회원의 닉네임	저장되고, 탈퇴 회원의 닉네임은 `deleted_user_{번호 10자리}`가 됩니다
동시에 바꾸기 (확인을 통과시켜 흉내 냄)	`commit`에서 UNIQUE 위반이 나서 409이고, 내 닉네임은 그대로입니다

전체 시험 256개가 통과했고, `ruff` 검사도 통과했습니다. 시험이 끝난 뒤 DB에는 데모 계정과 사용자 계정만 남아 있습니다.

Backend(FastAPI/Python) 38 — `save_profile_image()` (+ `image_extension()`)

항목	내용
파일	`studionx/backend/app/services/user_service.py`
작업	P1-08 회원 정보: 프로필 사진 올리기 (PRD 01 FR-106, jpg/png/webp, 2MB)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	37 `update_me()` · 39 `delete_profile_image()`

1. 하는 일

올라온 사진을 검사한 다음 `storage/profiles/{무작위 32자}.{확장자}`에 저장하고, 회원의 `profile_image_path`를 바꿉니다. 예전 사진 파일은 지웁니다.

올라온 파일	결과
JPG (1.2MB)	`profiles/3f9c….jpg`로 저장되고, 응답의 사진 주소도 바뀝니다
브라우저는 `image/png`라고 알렸지만 내용은 JPG	저장합니다. 확장자는 **내용을 기준으로** `.jpg`가 됩니다
GIF	415 `UNSUPPORTED_FILE_TYPE` "Use a JPG, PNG or WebP image."
`image/png`라고 알렸지만 내용이 HTML이거나 빈 파일	똑같이 415입니다
2MB를 1바이트라도 넘음	413 `FILE_TOO_LARGE` "Images must be 2 MB or smaller."
정확히 2MB	저장합니다
이미 사진이 있음	새 사진을 저장한 뒤 예전 파일을 지웁니다

오류가 나면 파일이 하나도 남지 않고, DB도 그대로입니다.

2. 언제 쓰나

쓰는 곳	
`POST /users/me/profile-image` (42)	`multipart/form-data`의 `file` 칸을 받아 이 함수를 부르고, 36번으로 응답합니다
화면	userInfo.html의 Upload photo입니다 (P1-10). 화면도 같은 종류와 크기를 먼저 검사합니다 (`PHOTO_TYPES`, `PHOTO_MAX_BYTES`)

용량: 프로필 사진은 저장 한도 5GB의 사용량에 **넣지 않습니다** (2026-10-07 사용자 결정). 그래서 `storage_used_bytes`를 건드리지 않습니다.

3. 코드

import (파일 맨 위)

```
from pathlib import Path, PurePosixPath
from uuid import uuid4
```

```
from fastapi import UploadFile
from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
from app.models import User
from app.schemas.user import MeResponse, MeUpdateRequest
from app.services.auth_service import check_nickname
```

`uuid4`와 `get_settings`를 이번에 추가했습니다.

도우미 함수 (내용으로 파일 종류 알아내기)

```
def image_extension(data: bytes) -> str | None:
    if data.startswith(b"\xff\xd8\xff"):
        return ".jpg"
    if data.startswith(b"\x89PNG\r\n\x1a\n"):
        return ".png"
    if data[:4] == b"RIFF" and data[8:12] == b"WEBP":
        return ".webp"
    return None
```

함수

```
async def save_profile_image(db: Session, user: User, file: UploadFile) -> User:
    if file.content_type not in PHOTO_TYPES:
        raise AppError(ErrorCode.UNSUPPORTED_FILE_TYPE, 415, "Use a JPG, PNG or WebP image.")
    data = await file.read(PHOTO_MAX_BYTES + 1)
    if len(data) > PHOTO_MAX_BYTES:
        raise AppError(ErrorCode.FILE_TOO_LARGE, 413, "Images must be 2 MB or smaller.")
    extension = image_extension(data)
    if extension is None:
        raise AppError(ErrorCode.UNSUPPORTED_FILE_TYPE, 415, "Use a JPG, PNG or WebP image.")

    storage_dir = get_settings().storage_dir
    relative = f"{PROFILE_DIR}/{uuid4().hex}{extension}"
    new_file = storage_dir / relative
    new_file.parent.mkdir(parents=True, exist_ok=True)
    new_file.write_bytes(data)

    old_relative = user.profile_image_path
    user.profile_image_path = relative
    try:
        db.commit()
    except Exception:
        db.rollback()
        new_file.unlink(missing_ok=True)
        raise
    if old_relative:
        (storage_dir / old_relative).unlink(missing_ok=True)
    db.refresh(user)
    return user
```

문법 풀이

import 문법	뜻
`from uuid import uuid4`	무작위 고유 번호를 만듭니다. `.hex`는 16진수 32자입니다
`from app.core.config import get_settings`	설정(`.env`)을 읽습니다. `storage_dir`는 `studionx/storage`(git에서 제외된 폴더)입니다

식	뜻
`async def`, `await file.read(n)`	파일을 **최대 n바이트까지만** 읽습니다. 업로드는 네트워크를 기다려야 하므로 FastAPI의 `UploadFile`은 비동기로 동작합니다
`b"\xff\xd8\xff"`	바이트 문자열입니다. `\xff`는 16진수 FF 한 바이트입니다
`data.startswith(…)`, `data[8:12]`	앞부분이 같은지 확인하고, 8번째부터 11번째 바이트를 꺼냅니다
`storage_dir / relative`	`Path`에서 `/`는 경로를 이어 붙입니다
`mkdir(parents=True, exist_ok=True)`	중간 폴더까지 만들고, 이미 있어도 오류를 내지 않습니다
`unlink(missing_ok=True)`	파일을 지우고, 파일이 없어도 오류를 내지 않습니다
`except Exception: … raise`	정리만 하고 **같은 오류를 다시** 던집니다 (오류를 삼키지 않습니다)

4. 흐름

1. 브라우저가 알려 준 종류를 확인하고, 허용하지 않는 종류면 415로 응답합니다.
2. 2MB + 1바이트까지만 읽고, 2MB를 넘으면 413으로 응답합니다.
3. 앞부분 바이트로 실제 종류를 확인하고, 아니면 415로 응답합니다. 확장자는 여기서 정합니다.
4. 무작위 이름으로 파일을 씁니다.
5. DB에 새 경로를 저장합니다. 실패하면 새 파일을 지우고 오류를 그대로 전달합니다.
6. 성공하면 예전 사진 파일을 지우고, DB에서 다시 읽어 돌려줍니다.

5. 왜 이렇게

선택	이유
종류를 **두 번** 확인 (알려 준 값과 실제 내용)	`content_type`은 브라우저(또는 공격자)가 마음대로 보낼 수 있습니다. HTML을 `image/png`라고 속여 저장시키면, 나중에 그 주소를 연 사람의 브라우저에서 스크립트가 실행될 위험이 있습니다
확장자를 **내용**으로 정함	파일을 돌려줄 때(40, 43) 확장자를 보고 종류를 정합니다. 그래서 내용과 확장자가 맞아야 합니다
`PHOTO_MAX_BYTES + 1`만 읽음	10GB짜리 파일을 올려도 2MB 남짓만 메모리에 올라옵니다. 1바이트를 더 읽어야 "딱 2MB"와 "2MB 초과"를 구분할 수 있습니다
저장 → DB → 예전 파일 삭제 순서 (템플릿은 "예전 파일 삭제 → 저장")	DB 저장이 실패해도 예전 사진이 남습니다. 템플릿 순서대로라면 실패했을 때 사진이 사라집니다. 시험으로 확인했습니다
이름은 무작위 32자 (회원 번호 없음)	35번의 사용자 결정입니다. 같은 회원이 사진을 바꾸면 이름도 바뀌어서 캐시 문제도 생기지 않습니다
`except Exception`으로 넓게 잡음	DB 연결이 끊기는 것처럼 어떤 오류가 나더라도 새 파일을 남기지 않기 위해서입니다. 오류는 다시 던져서 06번의 500 처리로 넘깁니다

선택	이유
그림 전체를 열어 보지는 않음	앞부분 바이트 검사로 종류는 확실히 가려낼 수 있습니다. 이미지 라이브러리(Pillow)로 다시 저장하면 더 안전하지만 패키지가 하나 늘어나므로, 필요해지면 나중에 추가합니다
사진 파일 응답에 `nosniff`	43번에서 `X-Content-Type-Options: nosniff`를 붙여서, 브라우저가 내용을 보고 종류를 바꿔 추측하지 않게 할 예정입니다

6. 시험 (`tests/test_user_service.py`에 17개를 추가해서 모두 39개, 실제 MySQL로 실행)

사진은 시험용 임시 폴더(`tmp_path`)에 저장했습니다. 실제 `storage/` 폴더는 건드리지 않았습니다.

시험	확인한 것
`image_extension` 7가지	jpg, png, webp는 인식하고, gif, HTML, WAV(RIFF이지만 WEBP가 아님), 빈 파일은 `None`입니다
JPG 저장	경로는 `profiles/{32자}.jpg`이고, 파일 내용이 같으며, 파일은 1개입니다. 응답 주소와 DB 경로가 바뀌고, `storage_used_bytes`는 0 그대로입니다
png라고 알렸지만 jpg	확장자가 `.jpg`입니다
GIF, HTML, 빈 파일, 2MB 초과	차례로 415, 415, 415, 413이고, 파일은 0개이며 경로는 그대로입니다
정확히 2MB	저장됩니다
사진 바꾸기	새 이름으로 저장되고, 예전 파일이 지워져서 파일은 1개입니다
예전 파일이 이미 없음	오류 없이 저장됩니다
DB 저장 실패	새 파일은 지워지고, 예전 사진 파일과 내용은 그대로이며, 오류가 그대로 전달됩니다

전체 시험 273개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 39 — `delete_profile_image()`

항목	내용
파일	`studionx/backend/app/services/user_service.py`
작업	P1-08 회원 정보: 프로필 사진 지우기 (PRD 01 FR-106)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	38 `save_profile_image()` · 40 `profile_image_file()`

1. 하는 일

내 프로필 사진을 지웁니다. DB의 `profile_image_path`를 비우고, 저장된 파일도 지웁니다. 그러면 화면은 닉네임 첫 글자가 들어간 동그라미로 돌아갑니다.

상황	결과
사진이 있음	경로는 `None`이 되고, 파일이 지워지며, 응답의 사진 주소는 `null`입니다
사진이 없음, 두 번 누름	오류 없이 `None` 그대로입니다
DB에는 경로가 있는데 파일은 이미 없음	오류 없이 경로를 `None`으로 바꿉니다
DB 저장 실패	오류를 그대로 전달하고, **파일은 남겨 둡니다**

2. 언제 쓰나

쓰는 곳	
`DELETE /users/me/profile-image` (42)	userInfo.html의 Remove 버튼(P1-10)을 누르면 이 함수를 부르고, 36번으로 응답합니다
P1-09 회원 탈퇴 `withdraw()`	템플릿 6번 "사진 파일 지우기 (delete_profile_image 재사용, commit은 마지막에 한 번)"에 해당합니다. 탈퇴 상태를 바꾼 **뒤 맨 마지막에** 이 함수를 부르면, 이 안의 `commit`이 탈퇴 상태와 사진 삭제를 **함께** 저장합니다

3. 코드

import (파일 맨 위: 38번과 같고, 새로 추가한 것은 없습니다). 이 함수는 **`get_settings`**를 사용합니다.

함수

```
def delete_profile_image(db: Session, user: User) -> User:
    old_relative = user.profile_image_path
    user.profile_image_path = None
    db.commit()
    if old_relative:
        (get_settings().storage_dir / old_relative).unlink(missing_ok=True)
    db.refresh(user)
    return user
```

문법 풀이

식	뜻
`old_relative = user.profile_image_path`	지우기 전에 경로를 따로 기억해 둡니다. 바로 아래에서 `None`으로 바꾸기 때문입니다
`get_settings().storage_dir / old_relative`	실제 파일의 위치입니다 (`studionx/storage/profiles/….jpg`)
`unlink(missing_ok=True)`	파일을 지우고, 이미 없어도 오류를 내지 않습니다
`db.refresh(user)`	DB가 바꾼 `updated_at`까지 다시 읽습니다

4. 흐름

1. 지금 경로를 기억해 두고 DB의 경로를 비웁니다.
2. 저장(commit)합니다. 실패하면 여기서 오류가 나고, 파일은 그대로 남습니다.
3. 저장에 성공했고 예전 경로가 있었다면 파일을 지웁니다.
4. 다시 읽어서 돌려줍니다.

5. 왜 이렇게

선택	이유
DB를 먼저, 파일은 나중에 처리 (템플릿은 "파일 삭제 → DB")	38번과 같은 이유입니다. 템플릿 순서대로라면 DB 저장이 실패했을 때, DB는 사진이 있다고 하는데 파일은 없어서 사진이 깨집니다. 이 순서라면 실패해도 사진이 그대로 보입니다 (반대로 파일 삭제가 실패하면 파일만 남을 뿐, 화면에는 영향이 없습니다)
오류 없이 끝나는 경우를 넓게 둠	Remove를 빠르게 두 번 누르거나, 파일을 손으로 지운 경우에도 사용자에게 오류를 보여 줄 이유가 없습니다
`commit`을 함수 안에서 함 (매개변수로 끄지 않음)	템플릿은 탈퇴에서 "commit은 마지막에 한 번"이라고 했습니다. 탈퇴가 이 함수를 **맨 마지막에** 부르면 같은 결과가 됩니다. 켜고 끄는 매개변수를 두면 "파일을 언제 지우는가"가 부르는 쪽의 책임이 되어 더 복잡해집니다
38번의 `rollback` 같은 정리 코드가 없음	지워야 할 새 파일이 없기 때문입니다. 오류가 나면 요청이 끝날 때 세션이 닫히면서 되돌려집니다

6. 시험 (`tests/test_user_service.py`에 5개를 추가해서 모두 44개, 실제 MySQL과 임시 폴더로 실행)

시험	확인한 것
사진 지우기	같은 객체를 돌려주고, 경로, 응답 주소, DB가 모두 `None`이며, 파일이 지워져 폴더가 비었습니다
사진 없음, 두 번 누름	오류가 없습니다
파일이 이미 없음	오류 없이 경로가 `None`이 됩니다
다른 회원의 사진	지워지지 않습니다
DB 저장 실패	오류가 그대로 전달되고, 파일과 내용은 그대로입니다

전체 시험 278개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 40 — `profile_image_file()`

항목	내용
파일	`studionx/backend/app/services/user_service.py`
작업	P1-08 회원 정보: 사진 파일 찾기 (PRD 01 FR-106)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	39 `delete_profile_image()` · 41 `GET · PATCH /users/me`

1. 하는 일

주소에 들어 있는 사진 이름(`3f9c….jpg`)으로 **디스크에 있는 실제 파일 경로**를 찾아 돌려줍니다. 보여 주면 안 되는 경우는 모두 똑같이 404로 응답합니다.

`name`	결과
활동 회원의 사진 이름	`C:/…/studionx/storage/profiles/3f9c….jpg`
정지(`SUSPENDED`)된 회원의 사진	경로를 돌려줍니다 (보입니다)
탈퇴(`DELETED`)한 회원의 사진 (3일 안이어도)	404
아무도 쓰지 않는 이름	404
DB에는 있는데 디스크에 파일이 없음	404
`../../.env`, 대문자, `.gif`, `….jpg.exe`, 31자, 빈 문자열	**DB를 보지 않고** 404

404는 모두 `NOT_FOUND` "Image not found."입니다. 어느 경우인지는 알려 주지 않습니다.

2. 언제 쓰나

쓰는 곳	
`GET /api/v1/profile-images/{name}` (43)	화면의 `<img src>`가 부르며, 로그인 없이 동작합니다 (`<img>`에는 토큰을 붙일 수 없습니다). 이 함수가 돌려준 경로를 `FileResponse`로 보냅니다

35번의 사용자 결정에 따라, 회원 번호가 아니라 **사진을 올릴 때 만든 무작위 이름**으로 찾습니다.

3. 코드

import (파일 맨 위)

```
import re
from pathlib import Path, PurePosixPath
from uuid import uuid4

from fastapi import UploadFile
from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session
```

```
from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
from app.models import User
from app.models.user import UserStatus
from app.schemas.user import MeResponse, MeUpdateRequest
from app.services.auth_service import check_nickname
```

`re`와 `UserStatus`를 이번에 추가했습니다.

상수 (이번에 추가)

```
# 주소로 받는 사진 이름 모양 — 올릴 때 만드는 uuid4().hex(소문자 16진수 32자) + 허용 확장자
PHOTO_NAME = re.compile(r"[0-9a-f]{32}\.(jpg|png|webp)")
```

함수

```
def profile_image_file(db: Session, name: str) -> Path:
    not_found = AppError(ErrorCode.NOT_FOUND, 404, "Image not found.")
    if not PHOTO_NAME.fullmatch(name):
        raise not_found
    user = db.scalar(select(User).where(User.profile_image_path == f"{PROFILE_DIR}/{name}"))
    if user is None or user.status == UserStatus.DELETED:
        raise not_found
    storage_dir = get_settings().storage_dir.resolve()
    path = (storage_dir / user.profile_image_path).resolve()
    if not path.is_relative_to(storage_dir) or not path.is_file():
        raise not_found
    return path
```

문법 풀이

import 문법	뜻
`import re`	정규식(글자 모양 검사)을 씁니다
`from app.models.user import UserStatus`	회원 상태 목록입니다 (P1-03 08)

식	뜻
`re.compile(r"…")`	정규식을 미리 한 번 만들어 둡니다. `r"…"`은 `\`를 그대로 쓰는 문자열입니다
`[0-9a-f]{32}`, `\.`, `(jpg|png|webp)`	차례로 소문자 16진수 32개, 점, 셋 중 하나를 뜻합니다
`PHOTO_NAME.fullmatch(name)`	**처음부터 끝까지** 모양이 맞아야 참입니다 (`….jpg.exe`는 거짓)
`.resolve()`	`..`과 바로 가기(심볼릭 링크)를 풀어서 진짜 절대 경로로 바꿉니다
`path.is_relative_to(storage_dir)`	경로가 그 폴더 **안에** 있는지 확인합니다 (Python 3.9 이상)
`path.is_file()`	디스크에 파일로 존재하는지 확인합니다

4. 흐름

1. 이름의 모양이 맞지 않으면 404로 응답합니다 (DB를 보지 않습니다).
2. 그 이름을 쓰는 회원을 찾습니다. 없거나 탈퇴한 회원이면 404로 응답합니다.
3. 실제 경로를 만들고, storage 폴더 안에 있는지와 파일이 있는지 확인합니다. 아니면 404로 응답합니다.
4. 경로를 돌려줍니다.

5. 왜 이렇게

선택	이유
이름 모양 검사를 **맨 앞**에 둠	`../../.env` 같은 이름은 DB를 보기도 전에 끝납니다. 모양이 맞는 이름만 경로가 될 수 있으므로, 폴더 밖으로 나가는 길이 처음부터 막힙니다
그래도 `is_relative_to`를 한 번 더 확인	템플릿이 요구한 사항입니다. DB 값이 어떤 이유로든(손으로 고치는 등) 이상해지더라도 storage 밖의 파일을 보내지 않습니다. 이중 잠금인 셈입니다
탈퇴 회원은 3일 안이어도 404	탈퇴는 "사이트에서 나를 지운다"는 뜻입니다. 파일 자체는 P1-09 탈퇴가 39번으로 지우지만, 그 전에도 보이지 않게 합니다. 추적 방지 ①과도 맞습니다
정지된 회원의 사진은 보임	정지는 관리자의 조치일 뿐이라서, 협업 화면(Phase 6)의 다른 멤버에게까지 사진을 숨길 이유가 없습니다. 바꾸려면 조건 한 줄만 고치면 됩니다
모든 404를 같은 문구로 응답	어떤 이름이 "있었는데 탈퇴했는지" 아니면 "처음부터 없었는지" 구분할 수 없게 합니다
소문자만 허용	`uuid4().hex`는 항상 소문자입니다. DB 비교는 대소문자를 무시하지만, 주소는 한 가지 모양만 받아서 같은 사진이 여러 주소로 열리지 않게 합니다
대문자나 형식 오류도 404 (422가 아님)	파일 주소이므로 "그런 파일이 없다"는 응답이 자연스럽습니다

알아 둘 점 (색인): `profile_image_path` 칸에는 색인(index)이 없어서, 사진을 열 때마다 회원 표를 처음부터 훑습니다. 회원이 수천 명 정도까지는 문제가 없습니다. 회원이 많아지면 `users.profile_image_path`에 UNIQUE 색인을 추가하면 됩니다 (DB.md 변경이 필요하므로 그때 사용자와 정합니다).

6. 시험 (`tests/test_user_service.py`에 12개를 추가해서 모두 56개, 실제 MySQL과 임시 폴더로 실행)

시험	확인한 것
활동 회원의 사진	storage 안의 절대 경로를 돌려주고, 파일 내용이 같습니다
이상한 이름 7가지	`../../.env`, 인코딩된 `..%2F`, 대문자, `.gif`, `.jpg.exe`, 31자, 빈 문자열은 모두 404입니다
아무도 쓰지 않는 이름	404
탈퇴한 회원	404
정지된 회원	경로를 돌려줍니다
디스크에 파일이 없음	404

모든 404가 `NOT_FOUND`와 "Image not found."로 같다는 것을 확인했습니다. 전체 시험 290개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 41 — `GET · PATCH /api/v1/users/me`

항목	내용
파일	`studionx/backend/app/api/v1/users.py`의 `read_me()`, `update_me()`
작업	P1-08 회원 정보: 내 정보 조회와 닉네임 변경 (PRD 01 FR-106)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	40 profile_image_file() · 42 POST · DELETE /users/me/profile-image

1. 하는 일

요청	응답
`GET /api/v1/users/me` + `Authorization: Bearer <Access>`	200과 내 정보 7칸 (36)
`PATCH /api/v1/users/me` + `{"nickname": " Route_New "}`	200과 바뀐 내 정보 (`"nickname": "Route_New"`)
`PATCH` + `{}`	200이고, 그대로입니다
`PATCH` + `{"email": "new@example.com"}`	200이고, 그대로입니다. 이메일은 바꿀 수 없으며(FR-106), 모르는 칸은 무시합니다
`PATCH` + `{"nickname": " a "}`, 21자	422 `VALIDATION_ERROR`와 `details.nickname`
`PATCH` + 다른 사람이 쓰는 닉네임	409 `NICKNAME_TAKEN`이고, 탈퇴한 지 3일이 안 된 닉네임이면 409 `REJOIN_TOO_SOON`입니다 (37)
토큰 없음, 틀린 토큰, 탈퇴 회원	401 `UNAUTHORIZED` (06 `CurrentUser`)

2. 언제 쓰나

화면 (JS)	부르는 것
userInfo.html의 `loadMe` (P1-10)	`GET`으로 닉네임, 이메일, 사진 칸을 채웁니다
userInfo.html의 닉네임 Save changes (P1-10)	`PATCH`
myPage.html (P1-17)	대시보드 API(P1-16)가 따로 있습니다. 이 API는 회원 정보 화면용입니다

함수들이 이어지는 순서 (`PATCH`)

순서	처리하는 곳	하는 일
①	06 `CurrentUser`	토큰으로 활동 회원을 찾습니다 (아니면 401)
②	`MeUpdateRequest` (37)	본문을 검사합니다. 공백을 지운 뒤 2~20자여야 합니다 (아니면 422, 03번 처리기)
③	37 `update_me`	중복과 탈퇴 3일 규칙을 확인하고 저장합니다 (아니면 409)
④	36 `to_me_response`	7칸 응답을 만듭니다

3. 코드

import (파일 맨 위)

```
from fastapi import APIRouter, Response, UploadFile, status
from fastapi.responses import FileResponse

from app.api.deps import CurrentUser, Db
from app.schemas.common import ErrorResponse
from app.schemas.user import (
    MeResponse,
    MeUpdateRequest,
    PasswordChangeRequest,
    WithdrawRequest,
)
from app.services import user_service
```

`user_service` 줄을 이번에 추가했습니다.

함수 (경로와 응답 설정은 템플릿 그대로입니다)

```
@router.get("/me", response_model=MeResponse, responses={401: {"model": ErrorResponse}})
def read_me(user: CurrentUser) -> MeResponse:
    return user_service.to_me_response(user)


@router.patch(
    "/me",
    response_model=MeResponse,
    responses={409: {"model": ErrorResponse}, 422: {"model": ErrorResponse}},
)
def update_me(data: MeUpdateRequest, user: CurrentUser, db: Db) -> MeResponse:
    return user_service.to_me_response(user_service.update_me(db, user, data))
```

문법 풀이

import 문법	뜻
`from app.services import user_service`	모듈 전체를 가져와서 `user_service.update_me`처럼 **어느 파일의 함수인지** 보이게 합니다 (라우터 함수 이름도 `update_me`라서 겹치지 않게 하기 위해서입니다)

식	뜻
`@router.get("/me", …)`, `@router.patch(…)`	이 라우터는 `/api/v1/users`에 붙어 있으므로 `GET · PATCH /api/v1/users/me`가 됩니다
`user: CurrentUser`	06번의 의존성입니다. 토큰을 검사해서 회원을 넣어 줍니다
`data: MeUpdateRequest`	JSON 본문을 이 모양으로 검사해서 넣어 줍니다. 틀리면 함수가 호출되지도 않고 422로 응답합니다
`db: Db`	요청마다 만들어지는 DB 세션입니다 (P1-03)
`response_model=MeResponse`	응답을 이 모양으로 한 번 더 걸러서 보내고, 문서(`/docs`)에도 표시합니다

4. 흐름

GET: 토큰을 확인하고 36번으로 응답합니다.

PATCH: 토큰을 확인하고, 본문을 검사한 뒤, 37번으로 바꾸고 36번으로 응답합니다.

5. 왜 이렇게

선택	이유
라우터는 한 줄이고, 실제 일은 서비스에 둠	P1-04, 05와 같은 원칙입니다. 규칙은 서비스에서 시험하고, 라우터는 연결만 합니다
모르는 칸(`email`)은 무시	pydantic의 기본 동작입니다. 이메일을 바꾸려는 요청도 오류 없이 "바뀌지 않음"으로 끝납니다. 화면은 이메일 칸을 읽기 전용으로 보여 줍니다
바뀐 정보를 응답으로 돌려줌	화면이 다시 `GET`을 보내지 않아도 됩니다
응답에 `user_id`가 있어도 괜찮음	토큰 주인 본인에게만 가기 때문입니다 (36 §2)

6. 시험 (`tests/test_user_routes.py`에 9개를 추가해서 모두 11개, 실제 앱과 MySQL로 실행)

시험	확인한 것
`GET`	200, 7칸, 값이 맞고, 사진은 `null`이며, 응답에 비밀번호 해시 문자열이 없습니다
`GET` 토큰 없음	401 `UNAUTHORIZED`
`PATCH` 새 닉네임	200 `Route_New`이고, DB에도 저장됩니다
`PATCH {}`	200이고, 그대로입니다
`PATCH` 이메일	200이고, 이메일은 그대로입니다
`PATCH " a "`, 21자	422 `VALIDATION_ERROR`이고, `details`에 `nickname`이 담깁니다
`PATCH` 다른 사람의 닉네임 (대소문자만 다름)	409 `NICKNAME_TAKEN`
`PATCH` 토큰 없음	401

전체 시험 299개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 42 — `POST · DELETE /api/v1/users/me/profile-image`

항목	내용
파일	`studionx/backend/app/api/v1/users.py`의 `upload_profile_image()`, `delete_profile_image()`
작업	P1-08 회원 정보: 프로필 사진 올리기와 지우기 (PRD 01 FR-106)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	41 `GET · PATCH /users/me` · 43 `GET /profile-images/{name}`

1. 하는 일

요청	응답
`POST` multipart, `file` 칸에 JPG(1.2MB)	200과 내 정보입니다. `"profile_image_url": "/api/v1/profile-images/3f9c….jpg"`
`POST` GIF	415 `UNSUPPORTED_FILE_TYPE`
`POST` 2MB 초과	413 `FILE_TOO_LARGE`
`POST` 칸 이름이 `file`이 아님 (`photo`)	422 `VALIDATION_ERROR`
`DELETE`	200과 내 정보입니다. `"profile_image_url": null`이고, 파일이 지워집니다
`DELETE` (사진 없음, 두 번 누름)	200
토큰 없음	401이고, 파일을 저장하지 않습니다

2. 언제 쓰나

화면 (userInfo.html, P1-10)	부르는 것
Upload photo → 파일 고르기	`FormData`에 `file`을 넣어 `api.post('/users/me/profile-image', formData)`로 보내고, 응답의 `profile_image_url`로 `<img>`를 바꿉니다
Remove	`api.delete('/users/me/profile-image')`로 보내고, 첫 글자 동그라미로 돌아갑니다

함수들이 이어지는 순서 (`POST`)

순서	처리하는 곳	하는 일
①	06 `CurrentUser`	토큰으로 활동 회원을 찾습니다 (아니면 401이고, 파일은 읽지도 않습니다)
②	FastAPI `UploadFile`	multipart의 `file` 칸을 받습니다 (없으면 422)
③	38 `save_profile_image`	종류, 크기, 내용을 검사하고 저장한 뒤 예전 파일을 지웁니다
④	36 `to_me_response`	새 사진 주소(35)가 담긴 응답을 만듭니다

3. 코드

import (파일 맨 위: 41번과 같고, 새로 추가한 것은 없습니다)

함수 (경로와 응답 설정은 템플릿 그대로입니다)

```
@router.post(
    "/me/profile-image",
    response_model=MeResponse,
    responses={413: {"model": ErrorResponse}, 415: {"model": ErrorResponse}},
)
async def upload_profile_image(file: UploadFile, user: CurrentUser, db: Db) -> MeResponse:
    user = await user_service.save_profile_image(db, user, file)
    return user_service.to_me_response(user)


@router.delete("/me/profile-image", response_model=MeResponse)
def delete_profile_image(user: CurrentUser, db: Db) -> MeResponse:
    return user_service.to_me_response(user_service.delete_profile_image(db, user))
```

문법 풀이

식	뜻
`file: UploadFile`	multipart 본문의 `file` 칸을 파일로 받습니다. 큰 파일은 디스크의 임시 파일에 두어서 메모리를 아낍니다. `python-multipart` 패키지가 필요합니다 (requirements에 있습니다)
`async def`, `await`	38번이 `await file.read(…)`를 쓰기 때문에 라우터도 비동기로 만듭니다
`user = await …`	매개변수 `user`에 저장 결과(같은 회원, 새 경로)를 다시 넣습니다

4. 흐름

POST: 토큰을 확인하고, `file` 칸을 받은 뒤, 38번으로 검사하고 저장한 다음 36번으로 응답합니다.

DELETE: 토큰을 확인하고, 39번으로 지운 뒤 36번으로 응답합니다.

5. 왜 이렇게

선택	이유
라우터는 연결만 함	검사, 저장 순서, 실패 처리는 38번과 39번에서 이미 시험을 마쳤습니다
응답은 내 정보 전체	화면이 새 사진 주소를 바로 받아서 `<img src>`를 바꿀 수 있습니다 (다시 `GET`을 보낼 필요가 없습니다)
사진이 없어도 `DELETE`는 200	39번과 같은 이유입니다. 두 번 눌러도 사용자에게 오류를 보여 주지 않습니다
토큰 검사를 파일보다 먼저 함	로그인하지 않은 사람이 파일을 올려도 저장되지 않습니다 (시험으로 확인했습니다: `profiles` 폴더가 생기지도 않습니다)

알아 둘 점 (비동기 함수 안의 DB 작업): `upload_profile_image`는 `async` 함수인데, 38번 안의 DB 저장(`commit`)은 동기 방식입니다. 그래서 저장하는 동안 서버의 다른 요청이 아주 잠깐 기다립니다. 사진 저장은 한 번에 수 밀리초 정도라서 Phase 1에서는 문제가 없습니다. 사용자가 많아지면 파일만 먼저 읽고, DB 작업은 동기 함수로 나누면 됩니다.

6. 시험 (`tests/test_user_routes.py`에 7개를 추가해서 모두 18개, 실제 앱과 MySQL, 임시 폴더로 실행)

시험	확인한 것
JPG 올리기	200이고, 주소 모양이 `/api/v1/profile-images/{32자}.jpg`이며, 그 파일의 내용이 올린 것과 같습니다
GIF, 2MB 초과	415 `UNSUPPORTED_FILE_TYPE`, 413 `FILE_TOO_LARGE`
칸 이름 `photo`	422 `VALIDATION_ERROR`
토큰 없이 올리기	401이고, 저장 폴더가 생기지도 않습니다
지우기	200이고, 주소는 `null`이며 파일이 지워집니다. 한 번 더 지워도 200입니다
토큰 없이 지우기	401

전체 시험 306개가 통과했고, `ruff` 검사도 통과했습니다. 실제 `storage/` 폴더는 비어 있습니다 (`.gitkeep`만 있습니다).

Backend(FastAPI/Python) 43 — `GET /api/v1/profile-images/{name}`

항목	내용
파일	`studionx/backend/app/api/v1/users.py`의 `images_router`, `read_profile_image()` / `app/api/v1/router.py` (한 줄)
작업	P1-08 회원 정보: 프로필 사진 파일 보내기 (PRD 01 FR-106)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	42 `POST · DELETE /users/me/profile-image` · 44 `check current password()` (P1-09)

1. 하는 일

`<img src="/api/v1/profile-images/3f9c….jpg">`가 부르는 주소입니다. 사진 **파일 자체**를 보내며, 로그인은 필요하지 않습니다.

요청	응답
올린 사진의 주소 (토큰 없이)	200과 그림 바이트입니다. 헤더는 `Content-Type: image/jpeg`(png, webp도 각각 맞게), `X-Content-Type-Options: nosniff`, `Cache-Control: public, max-age=86400`입니다
없는 이름, 모양이 맞지 않는 이름	404 `NOT_FOUND` "Image not found."
지운 사진, 탈퇴한 회원의 사진	똑같이 404입니다
`..%2F..%2F.env`	404입니다. `/`로 풀려서 이 주소와 맞지도 않습니다 (앱 공통 404)
템플릿의 `/api/v1/users/85/profile-image`	404입니다. **없는 주소**입니다

2. 언제 쓰나

쓰는 곳	
화면의 `<img src>`	userInfo.html과 myPage.html의 프로필 사진(P1-10, P1-17), 협업 멤버 사진(Phase 6)
주소를 얻는 곳	36번 응답의 `profile_image_url`입니다 (35번이 만듭니다). 화면은 받은 주소를 그대로 넣기만 합니다

사용자 결정 반영 (35 §2)

템플릿의 주소는 `GET /users/{user_id}/profile-image`였습니다. **회원 번호로 여는 공개 주소는 만들지 않는다**(탈퇴 추적 방지 ③)는 결정에 따라 `/profile-images/{무작위 이름}`으로 바꿨습니다. 이 주소는 `/users` 아래가 아니어서 라우터를 하나 더 두었습니다.

바꾼 곳	내용
`users.py`	`images_router = APIRouter()`와 그 위에 놓인 라우터 함수
`router.py`	`api_router.include_router(users.images_router, prefix="/profile-images", tags=["users"])`

바꾼 곳	내용
`tests/test_user_routes.py`	연결 확인 시험을 새 주소로 바꾸고, 옛 주소는 **없다**는 것을 확인합니다

3. 코드

import (파일 맨 위: 41번과 같고, 새로 추가한 것은 없습니다). 이 함수는 **`FileResponse`**와 40번 `profile_image_file`을 사용합니다.

라우터와 상수 (이번에 추가)

```
# 사진 파일 — /api/v1/profile-images/{name} (회원 번호가 아니라 무작위 파일 이름, 2026-10-07)
# /users 아래가 아니라서 라우터를 따로 둠 → router.py 에서 prefix="/profile-images" 로 붙임
images_router = APIRouter()

# 확장자 → 응답 종류 (PHOTO_TYPES 를 뒤집음: ".jpg" → "image/jpeg")
PHOTO_MEDIA_TYPES = {ext: media for media, ext in user_service.PHOTO_TYPES.items()}
```

함수

```
@images_router.get(
    "/{name}",
    response_class=FileResponse,
    responses={404: {"model": ErrorResponse}},
)
def read_profile_image(name: str, db: Db) -> FileResponse:
    path = user_service.profile_image_file(db, name)
    return FileResponse(
        path,
        media_type=PHOTO_MEDIA_TYPES[path.suffix],
        headers={"Cache-Control": "public, max-age=86400", "X-Content-Type-Options": "nosniff"},
    )
```

문법 풀이

식	뜻
`{ext: media for media, ext in ….items()}`	**사전을 만드는 식**입니다. `PHOTO_TYPES`(`"image/jpeg": ".jpg"`)의 키와 값을 뒤집은 사전을 만듭니다
`"/{name}"`, `name: str`	주소의 마지막 부분이 `name` 매개변수로 들어옵니다
`FileResponse(path, …)`	파일을 조금씩 읽어서 보냅니다 (통째로 메모리에 올리지 않습니다)
`path.suffix`	확장자 `".jpg"` 입니다
`response_class=FileResponse`	문서(`/docs`)에 "파일 응답"으로 표시합니다

4. 흐름

1. 40번으로 파일을 찾습니다. 보여 줄 수 없는 경우라면 40번에서 404로 끝납니다.
2. 확장자로 파일 종류를 정하고, 캐시 헤더와 `nosniff` 헤더를 붙여서 파일을 보냅니다.

5. 왜 이렇게

선택	이유
로그인 확인을 하지 않음	`<img>`는 `Authorization` 헤더를 붙일 수 없습니다. 대신 주소가 무작위 32자라서 이름을 모르면 열 수 없습니다 (2^{128}가지)
종류를 **확장자로** 직접 정함	38번이 내용을 보고 확장자를 붙였으므로, 확장자가 곧 실제 종류입니다. 운영체제의 종류 표에 기대지 않습니다 (Windows는 `.webp`를 모를 수 있습니다)
`X-Content-Type-Options: nosniff`	브라우저가 내용을 보고 "사실은 HTML이네" 하고 다르게 해석하지 않게 합니다. 38번의 내용 검사와 함께 이중 잠금이 됩니다
캐시는 1일 (`max-age=86400`, 템플릿 제안)	같은 사진을 매번 다시 받지 않습니다. 사진을 바꾸면 **이름이 바뀌므로** 새로 받습니다. 더 길게(1년) 할 수도 있지만, 지운 사진이나 탈퇴한 회원의 사진이 다른 사람의 브라우저에 오래 남지 않도록 하루로 정했습니다
`public`	사진은 이름만 알면 누구나 볼 수 있는 자료이므로, 공유 캐시(CDN)에 두어도 결과가 같습니다
라우터를 `users.py` 안에 둠	회원 사진은 회원 정보의 일부입니다. 파일을 새로 만들면 템플릿 머리말과 안내가 여러 곳으로 갈라집니다

6. 시험 (`tests/test_user_routes.py`에 8개를 추가해서 모두 26개, 실제 앱과 MySQL, 임시 폴더로 실행)

시험	확인한 것
jpg, png, webp를 올린 뒤 응답 주소로 열기 (토큰 없이)	200이고, 바이트가 같으며, `Content-Type`이 각각 맞고, `nosniff`와 캐시 헤더가 있습니다
없는 이름, `..%2F..%2F.env`, 모양이 맞지 않는 이름	404 `NOT_FOUND`
지운 사진, 탈퇴한 회원의 사진	404
옛 주소 `/users/{번호}/profile-image`	열리지 않습니다
연결 확인 (DB 없이)	`/api/v1/profile-images/{name}`은 있고, 옛 주소는 없습니다

처음에는 1개가 실패했습니다. `%2F`가 `/`로 풀리면서 사진 주소와 맞지 않아 이 함수까지 오지 않고, 앱 공통 404("Not found.")가 났기 때문입니다. 안전한 결과이므로, 그 경우는 상태와 코드만 확인하도록 시험을 고쳤습니다. 전체 시험 314개가 통과했고, `ruff` 검사도 통과했습니다. 실제 `storage/` 폴더는 비어 있습니다.